PEMROGRAMAN BERBASIS FRAMEWORK
JOBSHEET 10

NAMA : Muhammad Fahreza Rohmansyah
KELAS : 3E
NO. ABSEN : 15
NIM : 2341720151

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026

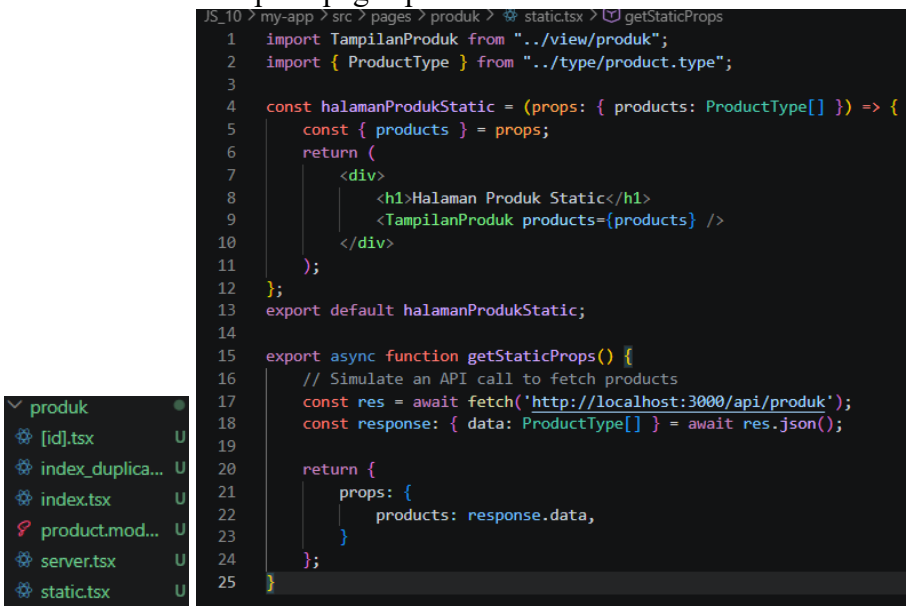
Link Github:

https://github.com/rezfahreza/PBF_2026.git

Praktikum

1. Setup Halaman Static

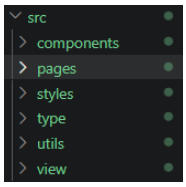
- Buat file baru pada pages/products/static.tsx lalu modifikasi static.tsx




```
import TampilanProduk from "../view/produk";
import { ProductType } from "../type/product.type";

const halamanProdukStatic = (props: { products: ProductType[] }) => {
    const { products } = props;
    return (
        <div>
            <h1>Halaman Produk Static</h1>
            <TampilanProduk products={products} />
        </div>
    );
};
export default halamanProdukStatic;

export async function getStaticProps() {
    // Simulate an API call to fetch products
    const res = await fetch('http://localhost:3000/api/produk');
    const response: { data: ProductType[] } = await res.json();

    return {
        props: {
            products: response.data,
        }
    };
}
```

2. Build Production Mode

- Memindahkan beberapa folder dari folder pages ke folder src



- Menjalankan npm run build, saat npm run dev sedang berjalan

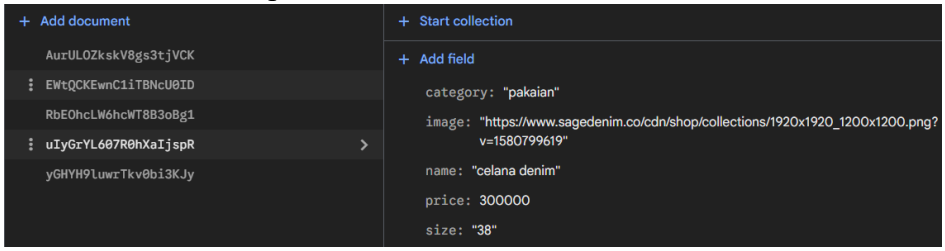
```
✓ Compiled successfully in 6.8s
✓ Collecting page data using 5 workers in 1271.9ms
✓ Generating static pages using 5 workers (18/18) in 636.2ms
✓ Finalizing page optimization in 20.1ms

Route (pages)
┌ ○ /
├   /_app
├ ○ /404
├ ○ /about
├ ƒ /api/hello
├ ƒ /api/produk
├ ○ /auth/login
├ ○ /auth/register
├ ○ /blog/[slug]
├ ○ /category/[...slug]
├ ○ /produk
├ ○ /produk/[id]
├ ○ /produk/index_duplicate
├ ƒ /produk/server
├ ● /produk/static (388 ms)
├ ○ /profile
├ ○ /profile/edit
├ ○ /profile
├ ○ /profile/edit
├ ○ /setting/app
├ ○ /shop/[[...slug]]
├ ○ /user
└ ○ /user/password
```

- Menjalankan npm run start, setelah mematikan npm run dev

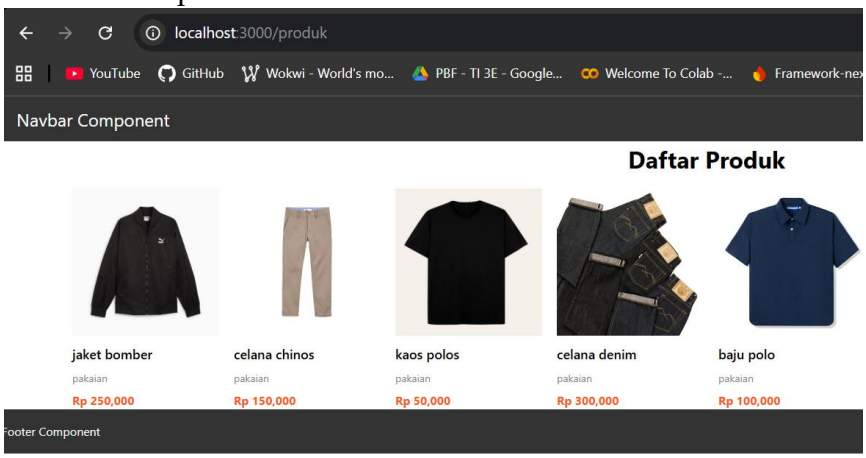
```
PS E:\PBF 2026\JS 10\my-app> npm run start

> my-app@0.1.0 start
> next start

▲ Next.js 16.1.6
- Local:         http://localhost:3000
- Network:       http://192.168.56.1:3000

✓ Starting...
✓ Ready in 766ms
```

3. Pengujian Perubahan Data

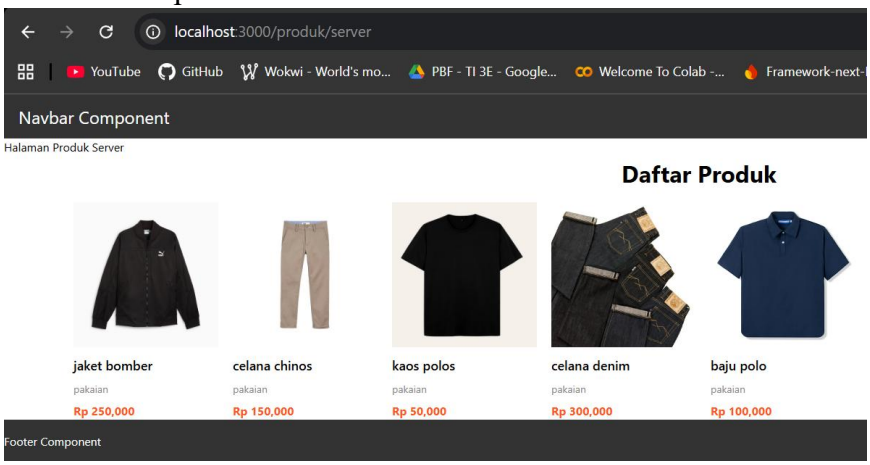
- Menambahkan produk



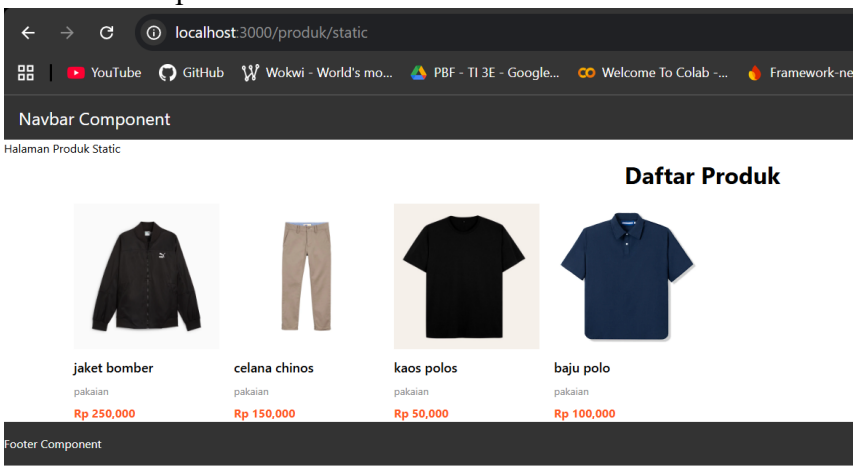
- ➢ CSR produk bertambah



- ➢ SSR produk bertambah



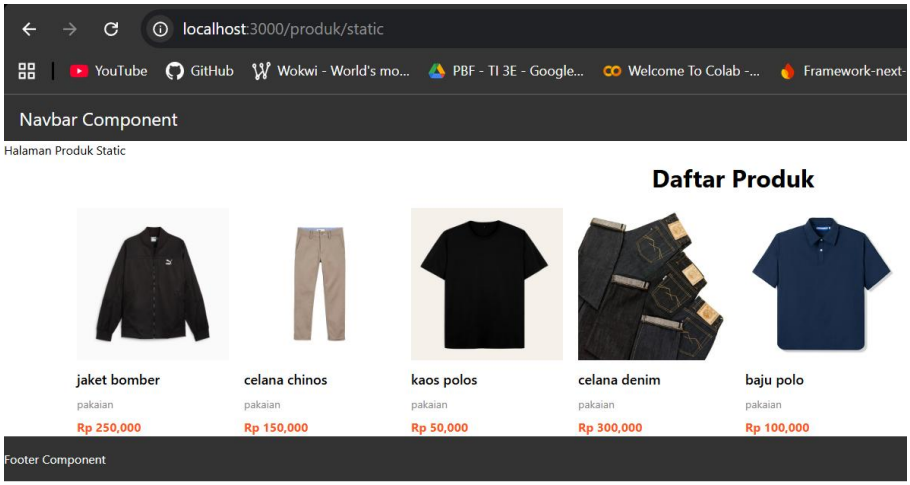
- ➢ SSG produk tidak bertambah



- Lakukan build ulang

```
PS E:\PBF_2026\JS_10\my-app> npm run build

✓ Compiled successfully in 7.9s
✓ Collecting page data using 5 workers in 1417.5ms
✓ Generating static pages using 5 workers (18/18) in 1779.4ms
✓ Finalizing page optimization in 64.0ms

Route (pages)
┌ o /
├   /_app
├ o /404
├ o /about
├ ƒ /api/hello
├ ƒ /api/produk
├ o /auth/login
├ o /auth/register
├ o /blog/[slug]
├ o /category/[...slug]
├ o /produk
├ o /produk/[id]
├ o /produk/index_duplicate
├ ƒ /produk/server
├ ● /produk/static (1541 ms)
├ o /profile
├ o /profile/edit
├ o /setting/app
├ o /shop/[[...slug]]
├ o /user
└ o /user/password

o  (Static)   prerendered as static content
●  (SSG)      prerendered as static HTML (uses getStaticProps)
ƒ  (Dynamic)  server-rendered on demand
```

- Run dan refresh halam static



Tugas Praktikum

1. Buat 3 halaman:

- CSG
- SSR
- SSG

CSR:

JS_9 > my-app > src > pages > produk > index.tsx > Kategori

```
import useSWR from "swr";
import styles from "./product.module.scss";
import fetcher from "../utils/swr/fetcher";
import TampilanProduk from "../view/produk";

type ProductType = {
  id: string;
  name: string;
  price: number;
  image: string;
  size: string;
  category: string;
};

const Kategori = () => {
  const { data, error, isLoading } = useSWR("/api/produk", fetcher);

  const products: ProductType[] = data?.data || [];

  if (error) {
    return <p>Gagal mengambil data</p>;
  }

  return (
    <TampilanProduk products={products} />
  );
};

export default Kategori;
```

SSR:

```
import TampilanProduk from "../view/produk";
import { ProductType } from "../../type/product.type";

const halamanProdukServer = (props: { products: ProductType[] }) => {
    const { products } = props;
    return (
        <div>
            <h1>Halaman Produk Server</h1>
            <TampilanProduk products={products} />
        </div>
    );
};
export default halamanProdukServer;

export async function getServerSideProps() {
    // Simulate an API call to fetch products
    const res = await fetch('http://localhost:3000/api/produk');
    const response = await res.json();

    return {
        props: {
            products: response.data, // Pass the products data to the page component
        }
    };
}
```

SSG:

JS_10 > my-app > src > pages > produk > static.tsx > getStaticProps

```
import TampilanProduk from "../view/produk";
import { ProductType } from "../type/product.type";

const halamanProdukStatic = (props: { products: ProductType[] }) => {
    const { products } = props;
    return (
        <div>
            <h1>Halaman Produk Static</h1>
            <TampilanProduk products={products} />
        </div>
    );
};
export default halamanProdukStatic;

export async function getStaticProps() {
    // Simulate an API call to fetch products
    const res = await fetch('http://localhost:3000/api/produk');
    const response: { data: ProductType[] } = await res.json();

    return {
        props: {
            products: response.data,
        }
    };
}
```

2. **Lakukan pengujian:**
- Tambah data
- Hapus data
- Bandingkan Hasil

Hasil dari ketiganya sama seperti pada saat pengujian perubahan data, untuk CSR dan SSR setelah terjadi perubahan data dapat langsung menampilkan data terbaru, beda dengan SSG

Pertanyaan Refleksi

1. Mengapa SSG tidak menampilkan data terbaru?
Jawab: SSG tidak menampilkan data terbaru karena proses pengambilan data (*fetching*) dan pembuatan HTML dilakukan pada saat build time. Data yang ditampilkan akan tetap sama dan tidak berubah sampai aplikasi dilakukan build ulang

2. Mengapa SSG lebih cepat?
Jawab: SSG lebih cepat karena halaman sudah di-*generate* menjadi file statis saat proses *build*. Ketika ada permintaan dari pengguna, server cukup mengirimkan file statis tersebut ke *browser* tanpa perlu melakukan proses *rendering* atau pengambilan data lagi di sisi server

3. Kapan SSG tidak cocok digunakan?
Jawab: SSG tidak cocok digunakan untuk halaman web yang memerlukan pembaruan data secara real-time atau dinamis. Jika konten situs sering berubah, penggunaan SSG akan tidak efisien karena setiap perubahan data mengharuskan pengembang untuk melakukan proses *build* ulang

4. Mengapa e-commerce tidak cocok menggunakan SSG murni?
Jawab: Aplikasi e-commerce biasanya memiliki data yang sangat dinamis, seperti perubahan stok atau harga barang yang terjadi setiap saat. Karena SSG bersifat statis hingga di-*build* ulang, metode ini kurang tepat; e-commerce lebih disarankan menggunakan SSR (Server Side Rendering) agar data selalu diperbarui pada setiap permintaan

5. Apa perbedaan build mode dan development mode?
Jawab:

- Development Mode (npm run dev): Digunakan selama proses pengembangan aplikasi agar pengembang dapat melihat perubahan kode secara langsung di lingkungan lokal
- Build Mode (npm run build): Proses untuk mengompilasi dan mengoptimasi aplikasi menjadi versi produksi, di mana pada metode SSG, tahap ini digunakan untuk mengambil data dan menghasilkan HTML statis